ARAŞTIRMA SORULARI

1.Kullanıcı,browser,fronted,backend,API,veritabanı ve Git/GitHub kavramları aynı projede nerede durur?

Bir kullanıcının ekrandaki bir butona tıklamasından verinin kalıcı olarak saklanmasına kadar uzanan bir zincirin halkalarıdır.

1.İstemci Tarafı(Client-Side)/Ön Yüz
-Kullanıcı(User):
–Sistemi kullanan,tarayıcı karşısındaki gerçek kişidir.Süreci başlatan girdileri(tıklama,form doldurma)sağlar.

-Tarayıcı(Browser):
–Web sitesinin üzerinde çalıştığı programdır(Chrome,Safari,Edge vb.)kodları görsel bir arayüze dönüştürür.

-Fronted:
–Kullanıcının tarayıcıda doğrudan gördüğü ve etkileşime girdiği her şeydir(HTML,CSS,JavaScript).
Örnek:Bir alışverişe sitesindeki"Satın Al"butonu,ürün resimleri ve sepet sayfası fronted'in parçasıdır.

2.İletişim Köprüsü:
-API(Application Programming Interface):
–Fronted ile backend arasındaki haberleşme köprüsüdür.Tarayıcıdan gelen istekleri arkaya taşır,arkadan gelen cevapları da öne iletir.
Örnek:Kullanıcı"Sepete Ekle"ye bastığında,fronted bir API isteği(request)oluşturarak arka plana"kullanıcı 5 nolu ürünü sepete eklemek istiyor"mesajını gönderir.

3.Sunucu Tarafı(Server-Side)/Arka Yüz
-backend:
–Kullanıcının görmediği,sistemin beynidir.Güvenlik kontrolü yapar,iş mantığını(logic)çalıştırır ve verileri işler.
Örnek:API'den gelen sepet isteğini alır;ürün stokta var mı,kullanıcı giriş yapmış mı gibi kontrolleri yapar.

-Veritabanı(Database):
–Projedeki tüm kalıcı verilerin(kullanıcı bilgileri,şifreler,ürünler,sipariş geçmişi)düzenli bir

şekilde saklandığı yerdir.backend,buraya veri yazar veya buradan veri okur.
Örnek:backend stok kontrolü için veritabanına bakar.Stok varsa,veritabanındaki ürün miktarını 1 azaltır ve kullanıcının sepetine bu ürünü kaydeder.

4.Projenin Yönetimi ve Depolanması(Versiyon Kontrolü):
-Git & GitHub:
–Bu kavramlar yukarıdaki akışın içinde değil,tüm bu sistemi geliştiren yazılımcıların kodlarını yönettiği yerdedir.

-Git:
–Yazılımcının kendi bilgisayarında çalışan,kodun geçmişini tutan ve yapılan değişiklikleri kaydeden bir versiyon kontrol sistemidir.

-GitHub:
–Git ile kaydedilen bu kodların ibternet üzerinde(bulutta)saklandığı,fronted ve backend kodlarının depolar(repository)halinde durduğu ve ekiplerin birlikte öaılşmasını sağlayan platformdur.

2.Bir kullanıcı butona bastığında veri hangi aşamalardan geçebilir?

1.İstemci(Client)Tarafı:Tetiklenme ve Hazırlık:
-Olay Yakalama(Event Listener):
–Kullanıcı butona bastığında(click-tap),yazılımdaki"Olay Dinleyici"bu hareketi algılar.

-Veri Toplama ve Doğrulama(Validation):
–Eğer buton bir form gönderiyorsa(örneğin giriş yapma veya kayıt olma),girilen veriler(e-posta,şifre vb.)toplanır.Boş alan var mı,e-posta formatı doğru mu gibi ilk kontroller(Client-Side Validation)burada yapılır.

-Paketleme(Serialization):
–Toplanan veriler,internet üzerinden taşınabilecek standart bir formata(genellikle JSON veya form-data)dönüştürülür.

2.Ağ(Network)Tarafı:Taşınma:
-İstek Oluşturma(HTTP request):
–Tarayıcı,sunucuya göndermek üzere bir HTTP isteği(çoğunlukla veri göndermek için POST veya PUT metodunu kullanır.)hazırlar.

-Güvenlik Katmanı(SSL/TLS):
–Veriler,yolda kötü niyetli kişiler tarafından ele geçirilmesin diye şifrelenir(HTTPS protokolü).

-Yönlendirme(Routing&API):
–Veri paketi internet kabloları,baz istasyonları ve sunucu önündeki yük dengeleyiciler(Load Balancer)üzerinden geçerek doğru uygulamanın kapısını çalar(API Endpoint-örn:/api/login).

3.Sunucu(Backend)Tarafı:İşleme
-Kimlik ve Yetki Kontrolü(authentication/Aunthorization):
–Sunucu ilk olarak "Bu isteği gönderen kişi kim ve bu butona basmaya yetkisi var mı?"sorusunu kontrol eder.

-Sunucu Tarafı Doğrulama(Server-Side Validation):
–İstemci tarafındaki güvenlik kontrolleri atlanabileceği için sunucu veriyi tekrar ve daha sıkı bi kontrolden geçirir.

-İş Mantığı(Business Logic):
–Uygulamanın asıl kuralları çalıştırılır.Örneğin;bir e-ticaret sitesinde "Satın Al"butonuna basıldıysa,stok kontrol edilir,fiyat hesaplanır ve ödeme entegrasyonu çağrılır.

4.Veri Katmanı(Database):Kalıcı Hale Getirme
-Sorgu Oluşturma:
–Sunucu,veri tabanının anlayacağı dilde bir sorgu(SQL veya NoSQL)hazırlar.

-Kayıt ve Güncelleme:
–Veri tabanına yeni bir satır eklenir(örn:yeni sipariş)veya mevcut veri güncellenir(örn:stok miktarı 1 azaltılır).

-Onay(Commit):İşlem başarıyla tamamlandığında veri tabanı sunucuya "İşlem başarılı"sinyali gönderir.

5.Geri Dönüş ve Yanıt(Response):
–Yanıt Paketleme:
–Sunucu,işlem sonucunu içeren bir HTTP yanıt kodu(örn:Başarılı için 200 OK,Hata için 400 Bad Request)ve bir veri paketi hazırlar.

-Ekrana Yansıtma(Rendering/UI Update):
–Ağ üzerinden geri gelen yanıtı alan tarayıcı veya mobil uygulama,arayüzü günceller.Kullanıcı ekranda"İşlem Başarılı"animasyonu,yeşil bir tik veya yeni bir sayfa görür.

3.Uygulamanın çalışan hali ile kaynak kodunun GitHub’daki hali aynı şey midir?

Hayır,uygulamanın çalışan hali(executable/runtime)ile GitHub'daki kaynak kodu(source code)aynı şey değildir.

Kaynak Kod(GitHub'daki Hal):
-Nedir?
–yazılımcıların Python,Java,C# veya JavaScript gibi programlama dilleriyle yazdığı,insanların okuyup anlayabileceği metin dosyalarından oluşur.

-Nerede Durur?
–GitHub gibi platformlar,bu kodların kaybolmaması ve birden fazla kişinin aynı anda kod üzerinde çalışabilmesi için bir bulut deposu(repository)görevi görür.

-Çalışır mı?
–GitHub'daki kod kendi başına bilgisayarınızda veya telefonunuzda doğrudan tıklayıp çalıştırabileceğiniz bir"program" değildir.

Çalışan Hali(Üretilen Hal):
-Nedir?
–Kaynak kodun,bilgisayarların veya telefonların(işlemcilerin)anlayabileceği 0 ve 1'lerden oluşan makine diline dönüştürülmüş halidir.Telefonunuzdaki bir .apk/.app dosyası veya bilgisayarınızdaki bir .exe dosyası uygulamanın çalışan halidir.

-Nasıl Oluşur?
–Kodlar bir derleyici(Comoiller)veya yorumlayıcı(ınterpreter)aracılığıyla işlenir.Bu süreçte kodlar sıkıştırılır,optimize edilir ve son kullanıcının açıp kullanabileceği bir paket haline getirilir.

4.Hata çıktığında problemi fronted,backend,Network veya veritabanı tarafında nasıl ayırmaya başlayabilirsiniz?

Frontend(Ön Yüz)Kaynaklı Hatalar:
Eğer sorun sadece kullanıcnın tarayıcısında(Arayüzde)yaşanıyorsa ve suncuya hiç ulaşmıyorsa frontend katmana odaklanmalısınız.
-Nasıl Anlaşılır?
–Tarayıcı konsolunda(console)kırmızı renkli JavaScript hata mesajları(Uncaught Typeerror,Cannot read property of undefind vb.)görürsünüz.
–Butonlara tıklandığında hiçbir tepki verilmez ancak Network sekmesinde sunucuya giden yeni bir istek(request)oluşmaz.
–Sayfa düzeni kaymıştır veya CSS/HTML yüklenme hataları vardır.
–İlk bakılcak yer:Tarayıcı konsolu(Console)ve kaynak kodlar(Sources).

Network(Ağ)Kaynaklı Hatalar:
İstek tarayıcıdan çıkmış ancak backend sunucusuna hiç ulaşamamış veya yolda engellenmiş olabilir.
-Naıl çalışır?
–Tarayıcı Hataları:
-ERR_CONNECTION_TIMED_OUT,ERR_NAME_NOT_RESOLVED(DNS sorunu)veya ERR_CONNECTION_REFUSED gibi hatalar alınır.

–CORS Hataları:
-Konsolda Cross-Origin Request Blocked uyarısı çıkar.Bu durum,frontend ile backend'in güvenli haberleşemediğini gösterir.

–Durum Kodları(Status Codes):
-İsteklerin Network sekmesinde kırmızı renkte kalması veya Status:(failed)yazması.

–İlk Bakılcak yer
-Tarayıcı Network sekmesi,DNS ayarları,VPN/Proxy durumu ve SSL sertifikaları.

Backend(Arka Yüz)Kaynaklı Hatalar:
Fronted sunucuya başarıyla istek göndermiş,ancak sunucu bu isteği işlerken bir probleme karşılaşmıştır.
-Nasıl Anlaşılır?
–Network sekmesindeki API istekleri 5xx(Sunucu Hataları)veya 4xx(İstemci Hataları)durum kodları döner.
–500 Internal Server Error:
-Backend kodundaki bir çökme(crash),null pointer exception veya mantık hatası olduğunu gösterir.
–502 Bad Gateway/504 Gateway Timeout:
-Nginx/Apache gibi bir ters proxy sunucusunun arkadaki backend uygulamasına ulaşamadığını veya uygulamanın çok geç yanıt verdiğini gösterir.
–400 Bad Request/422 Unprocessable Entity:
-Frontedn'in gönderdiği verinin backend'in beklediği formatta olmadığını gösterir.
–İlk Bakılacak Yer:
-Backend uygulama logları(Application Logs)ve Network sekmesindeki ilgili isteğin Response(Yanıt)içeriği.

Veritabanı(Database)Kaynaklı Hatalar:
Bcakend ayaktadır ancak bilgi okumaya veya yazmaya çalışırken veritabanı seviyesinde bir tıkınma veya hata yaşanıyordur.
-Nasıl Anlaşılır?
–Backend loglarında Connection Timeout,Deadlock detected,Sytanx error in SQL veya Max Connections reached gibi hata mesajları görünür.

–Network sekmesindeki API istekleri 500 veya 504 döber ve bu istelerin yanıt süresi(Time to First Byte-TTFB)çok uzundur(sunucu veritabanını beklediği için).
–Uygulamada listeleme,giriş yapma veya kaydetme gibi veriye dayalı işlemler çalışmazken,statik sayfalar sorunsuz açılır.
–İlk Bakılcak Yer:Veritabanı logları,aktif sorgu yoğunluğu(CPU/Memory kullanımı)ve backend-database bağlantı havuzu(conncetion pool).

5.Bu dört haftada öğrendiğiniz kavramlardan hangileri birbirine bağlı çıktı?

-C#,.NET ve Visual Studio:
–C# ile kod yazılır,.NET kodun çalışmasını sağlar,Visual Studio ise kod yazmak için kullanılır.

-CPU,RAM ve Program:
–Bir programın bilgisayarda nasıl çalıştığını ve verilerin nerede tutulduğunu anlamamı sağladı.

-Değişkenler,Array ve List:
–Program içerisinde verilerin tutulması,eklenmesi,okunması ve silinmesiyle bağlantılıdır.

-Frontend,Backend ve Server:
–Bir web sitesinde kullanıcının yaptığı işlemlerin sunucuya nasıl ulaştığını açıklar.

-SQL ve Veritabanı:
–Kullanıcı ve uygulama bilgilerinin düzenli şekilde saklanmasını sağlar.

-Git ve GitHub:
–Yazdığım kodların ve araştırma notlarımın saklanmasını,değişikliklerin takip edilmesini sağlar.

-Algoritma,Class ve Metot:
–Bir problemi küçük parçalara ayırarak daha düzenli çözümler oluşturmamı sağlar.

-authentication ve Aunthorization:
–Kullanıicnın kimliğinin doğrulanması ve hangi işlemleri yapabileceğinin belirlenmesiyle ilgilidir.

-SQL Injection ve XSS:
–Web uygulamalarında güvenliğin neden önemli olduğunu gösterir.

SONUÇ:
–Bu kavramların birbirinden bağımsız olmadığını,bir web uygulamasının

oluşturulması,çalışması,verilerin saklanması,hataların bulunması ve güvenliğin sağlanması için birlikte kullanıldığını öğrendim.

GÜN SONUNDA TESLİM EDECEĞİNİZ NOT:

1.Final notu hazırlayın:basit bir web uygulamasını kod yazmadan baştan sona tasarlayın.

1.Uygulamanın Amacı:
–Görev Takip Uygulaması,kullanıcıların günlük yapacakları işleri kaydetmelerini,görevlerie tarih belirlemelerini ve tamamladıkları işleri takip etmelerini sağlayan basit bir web uygulamasıdır.

2.Uygulamada Bulunacak Bölümler:
-Giriş Sayfası:
–Kullanıcı adı ve şifre ile giriş yapılır.
-Ana Sayfa:
–Kullanıcının görevleri görüntülenir.
-Görev Ekleme:
–Yeni görev oluşturulur.
-Görev Düzenleme:
–Eklenen görevler güncellenir.
-Görev Silme:
–İstenmeyen görevler silinir.
-Görev Tamamlama:
–Bitirilen görevler tamamlandı olarak işaretlenir.

3.Kullanılacak Bilgiler:
-Kullanıcı adı
-E-posta adresi
-şifre
-Görev başlığı
-Görev açıklaması
-Görev oluşturma tarihi
-Son teslim tarihi
-Görev durumu

4.Kullanılcak teknolojiler:
TEKNOLOJİ GÖREVİ
HTML : Web sayfasının yapısını oluşturur.
CSS : Sayfanın görünümünü düzenler.

JavaScript : Sayfadaki işlemleri gerçekleştirir.
C#/.NET : Sunucu tarafındaki işlemleri yönetir.
SQL : Görev ve kullanıcı bilgilerini saklar.
Git/GitHub : Kodları ve değişiklileri takip eder.

5.Uygulamanın Çalışma Aşamaları:



6.Veritabanı Tasarımı
Uygulamada iki tablo bulunacaktır:
-Kullanıcılar Tablosu:
–Kullanıcı ID
–Kullanıcı adı
–E-posta
–Şifre özeti(hash)

-Görevler Tablosu:
–Görev ID
–Kullanıcı ID
–Görev başlığı
–Açıklama
–Teslim tarihi
–Görev durumu

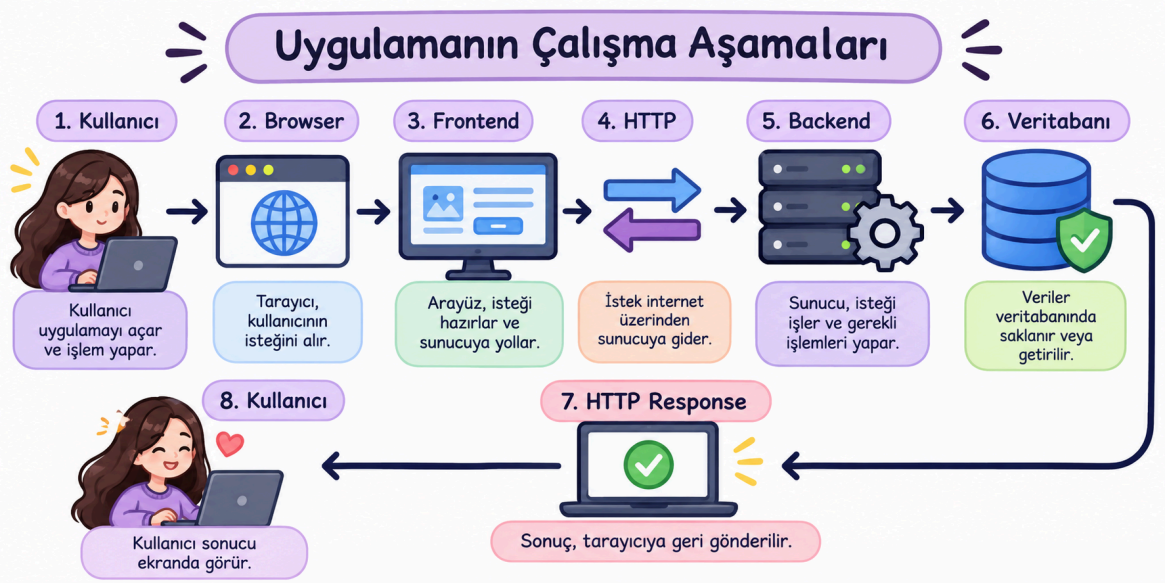
7.Güvenlik Önlemleri:
-Şifreler açık şekilde saklanmamalıdır.
-Kullanıcı girişi kontrol edilmelidir.
-Kullanıcılar yalnızca kendi görevlerine erişebilmelidir.

-SQL Injection saldırılarına karşı önlem alınmalıdır.
-XSS saldırılarına karşı kullanıcıdan gelen veriler güvenli işlenmelidir.
-HTTPS kullanılmalıdır.

8.Hata Kontrolü:
-Kullanıcı yanlış şifre girerse hata mesajı gösterilir.
-Görev başlığı boş bırakılırsa uyarı verilir.
-İstenen görev bulunamazsa bilgi verilir.
-Sunucu çalışmazsa hata mesajı gösterilir.

9.Git ve GitHub Kullanmı:
Uygulamanın geliştirme sürecinde Git ve GitHub kullanılır.
-Repository oluşturulur.
-Dosyalar düzenli klasörlere ayrılır.
-Yapılan değişiklikler commit edilir.
-Commit işlemleri push ile GiyHub'a gönderilir.
-Gizli bilgiler repository dışında tutulur.

2.Kullanıcı -> browser -> frontend -> HTTP -> backend -> veritabanı -> response akışını kendi çiziminizle gösterin.



3.Bu dört haftanın başında yanlış bildiğiniz veya hiç bilmediğiniz en az beş şeyi yazın:

1-Git ve GitHub

2-Client ve Server

3-Frontend ve Backend

4-authentication ve Aunthorization

5-Class ve Metot